Automated CIS Benchmark Remediation
using a LLM Pipeline

Pranjal Garg
IIIT Hyderabad

Contents

1 Problem Statement

Hardening Linux servers against CIS Benchmarks currently relies heavily on manual, unscalable human judgement. This project addresses the gap between context-blind vulnerability scanning and context-aware remediation.

- **The Context Problem:** OpenSCAP flags security issues, but it can't tell if an issue actually matters for that specific machine. Since a public server and a personal laptop have very different security needs, a human expert still has to review every single rule by hand.
- **The LLM Solution:** This project offloads that decision to an automated, role-aware LLM pipeline. Given an OpenSCAP scan and a role description, the agent decides which rules to fix, applies and verifies them, and only escalates to a human when it cannot proceed safely.
- **Project Execution:** The work was broken into three stages: building a trustworthy ground-truth answer key of keep-or-skip decisions, benchmarking candidate models to find the best fit, and developing a multi-step remediation agent (validated across multiple Ubuntu versions).
- **The Target Gap:** A baseline Ubuntu 24.04 scan yields a 62.39% compliance score (184 pass, 115 fail, 10 NA). Blindly applying OpenSCAP's default fixes raises this to 89.49% (290 pass, 9 fail, 7 NA). This project targets that remaining $\sim$11%—the complex, context-dependent rules where blind automation fails and an intelligent agent is actually required.

2 Setup: Profiles, Roles, and Models

A few things stay constant across every stage of this report.

2.1 Profiles and roles tested

Profile	Ubuntu version	Role description
MIN	24.04	A personal laptop used only by its owner, on a home or otherwise private and trusted network. Very few CIS rules genuinely apply here.
MAX	24.04	A system administrator running production infrastructure on a cloud platform like AWS. Almost every rule applies, and the security requirements are stricter.
Dev (validation)	22.04	A software developer working with containerised dev tools and a local server setup. Used purely to check the model choice generalises.

MIN and MAX are deliberately opposite ends of the profiles – one lenient, one strict – so a model's

judgement gets checked at both extremes rather than against a single assumed use case. The Software Developer profile was used later, on a different Ubuntu release, purely as a sanity check.

2.2 Models screened initially

Eight candidate models were run locally on a lab server and screened against the MIN/MAX ground truth before anything else happened:

Model	Hosting
DeepSeek-R1 (7B)	Local
Gemma 2 (latest)	Local
GPT-OSS (latest)	Local
Granite 4.1 (8B)	Local
Llama 3.2 (latest)	Local
Mistral (latest)	Local
Phi-3 (latest)	Local
Qwen 2.5 (7B)	Local

Every model was run entirely offline on the lab server so cost, latency, and the ability to run without internet access could be weighed alongside raw accuracy from the start.

3 Building a Ground Truth

To accurately evaluate model performance, I first established a reliable answer key of correct keep-or-skip decisions to serve as the benchmark.

- **Establishing the Answer Key:** We developed a primary ground truth for Ubuntu 24.04 that maps 63 CIS rules across 41 distinct role/profile combinations, dictating whether each rule should be kept and remediated, or safely skipped.
- **Rigorous Validation:** To ensure accuracy and eliminate bias, the ground truth decisions were cross-verified manually as well as against multiple LLMs (Claude, ChatGPT, and Gemini) before finalization.
- **Testing at the Extremes:** The core benchmarking on Ubuntu 24.04 deliberately targeted the MIN (highly lenient) and MAX (highly strict) profiles. This tested the models' judgement across the full spectrum of use cases rather than a single, average profile summary.
- **Cross-Version Generalization:** To prove the pipeline wasn't simply limited to our test environment, I built a secondary ground truth covering 48 rules for a Software Developer profile on Ubuntu 22.04, ensuring the final model choice holds up on untuned systems and roles.

4 Screening Eight Candidate Models

With the ground truth established, the core question, given a failing rule and a role description, should we keep or skip? - was put to the candidate models.

- **The Classification Task:** Eight models were tasked with classifying the same 63 failing rules against both the MIN and MAX ground truths to test their baseline judgement.
- **Scoring the MAX Profile:** The MAX ground truth is unanimous, every single one of the 63 rules is a KEEP. Therefore, a model's MAX accuracy is simply its keep-count out of 63. Any SKIP or failure to parse is unambiguously wrong.
- **Scoring the MIN Profile:** The MIN ground truth consists of a genuine mix of decisions (51 KEEP, 12 SKIP).
- **Phase Objective:** This first pass was deliberately broad. The sole goal was to eliminate weak performers and build a shortlist of models capable of moving on to the much harder task: proposing actual remediation scripts.

Model	MAX accuracy	MIN accuracy	Overall
Qwen 2.5 (7B)	98.4% (62/63)	82.5% (52/63)	90.5%
GPT-OSS	95.2% (60/63)	76.2% (48/63)	85.7%
Granite 4.1 (8B)	93.7% (59/63)	73.0% (46/63)	83.3%
Phi-3	54.0% (34/63)	68.3% (43/63)	61.1%
Gemma 2	100.0% (63/63)	66.7%† (42/63)	83.3%†
Mistral	98.4% (62/63)	81.0% (51/63)	89.7%
Llama 3.2	77.8% (49/63)	81.0% (51/63)	79.4%
DeepSeek-R1 (7B)	66.7% (42/63)	49.2% (31/63)	57.9%

Table 1: Screening accuracy for eight candidate models

- **Resource Utilization:** While accuracy was similar among the top models, timing varied. Note that the CPU time shown is just for the local test script, not the AI model itself. Because the heavy lifting was done on a remote lab server, local computer did very little work (using only $\sim$55 MB of RAM).

Model	MAX wall time	MIN wall time	MAX CPU time	MIN CPU time
Qwen 2.5 (7B)	8.8 min	8.7 min	3.03 s	1.68 s
GPT-OSS	41.0 min	44.8 min	3.04 s	1.91 s
Granite 4.1 (8B)	16.5 min	19.3 min	3.02 s	2.64 s
Phi-3	9.7 min	7.3 min	3.01 s	2.63 s
Gemma 2	17.1 min	25.0 min†	3.01 s	2.27 s
Mistral	12.3 min	12.2 min	3.13 s	3.56 s
Llama 3.2	4.6 min	4.5 min	3.04 s	4.05 s
DeepSeek-R1 (7B)	51.8 min	54.5 min	3.08 s	3.17 s

Table 2: Wall-clock and CPU time per model, per profile, across all 63 rules.

5 Shortlisting Models for a One-Pass Remediation Test

Five of the nine models advanced to the second stage. Here, the task shifted from simply deciding whether to keep or skip a rule to actually writing the script to fix it.

- **The Task:** Each model was given the failure details straight from the OpenSCAP scan and asked to propose a fix. For this stage, there were no second chances or retry loops, it was strictly a single-pass test. Every proposed fix was reviewed by a human before being applied.
- **Fair Benchmarking:** To ensure every model started from the exact same baseline, snapshots were used to reset any fixed rules back to their original broken state between runs.
- **Efficient Testing Strategy:** To avoid redundant testing, I first tested the 51 MIN rules. Then, for the MAX profile, we only tested the remaining 12 rules (which were in SKIP category for the other profile).

Model	MIN: fixed (51 rules)	MAX: fixed (12 rules)	Overall
Qwen 2.5 (7B)	40/51 (78.4%)	2/12 (16.7%)	42/63 (66.7%)
Gemma 2	38/51 (74.5%)	3/12 (25.0%)	41/63 (65.1%)
GPT-OSS	24/51 (47.1%)	1/12 (8.3%)	25/63 (39.7%)
Mistral	17/51 (33.3%)	1/12 (8.3%)	18/63 (28.6%)
Granite 4.1 (8B)	7/51 (13.7%)	3/12 (25.0%)	10/63 (15.9%)

Table 3: One-pass remediation results. "Fixed" means the rule came back `oscap_pass` after the model's proposed script was applied.

- **Final Selection:** Qwen 2.5 (7B) led the performance on the MIN profile, with Gemma 2 close behind. Ultimately, Qwen 2.5 (7B) was chosen for the the final pipeline.

6 Turning the Pipeline into a Multi-Step Agent

To move beyond single-shot proposals, the pipeline was redesigned into an autonomous agent capable of diagnosing and correcting its own mistakes.

- **Iterative Self-Correction:** Instead of stopping after one attempt, the agent cycles through a propose-apply-rescan loop (up to 3 attempts). If a fix fails, the exact error output is fed back to the model so it can adjust its approach for the next attempt.
- **Running Memory:** The agent is seeded with a few pre-verified fixes and known system-specific hints for certain rules, giving it extra context to draw on directly.
- **Human-in-the-Loop Escalation:** The agent pauses and requests human input in three specific scenarios:
  - *Contextual Ambiguity:* When a rule's application to a specific role is genuinely subjective, it pauses to ask the user targeted multiple-choice or free-form questions.
  - *Privileged Access:* When a fix requires interactive administrative credentials that the agent cannot generate itself.

- *High-Risk Changes:* Fixes altering firewalls or remote-access protocols always require explicit user confirmation, even when the agent is operating in fully autonomous mode.

7 Validating on a Second Ubuntu Release

To ensure the pipeline and model selection weren't simply limited to Ubuntu 24.04, the entire process was tested on an Ubuntu 22.04 machine using the Software Developer profile.

- **Classification Performance:** Out of the 49 rules evaluated, the pipeline correctly recommended remediating 47 rules, and accurately judged the remaining 2 filesystem kernel-module rules as safe to skip for a developer profile. This proved the model generalizes well to new releases without requiring a full re-screening.

Outcome (Ubuntu 22.04 validation run)	Count
Rules evaluated	49
Recommended to keep and remediate	47
Recommended to skip	2

Table 4: Final classification outcome for the Ubuntu 22.04 validation profile.

- **Live Remediation Success:** The pipeline attempted to apply fixes for 44 of the recommended-keep rules on the live machine. Ultimately, 36 rules (81.8%) successfully achieved an `oscap_pass`.
- **Handling Edge Cases:** The agent successfully adapted to environment quirks. For example, three cron-permission rules initially failed because `/etc/cron.allow` didn't exist on the dev machine. The agent successfully passed them once the script was dynamically adjusted to handle the missing-file case gracefully.
- **Persistent Failures:** Only three rules genuinely failed to resolve after all multi-step retry attempts (including password-hashing and the two GRUB rules).

8 Challenges Faced

During testing, several rule failures were caused by the environment or upstream source material, rather than poor model logic:

- **Missing Dependencies & Timeouts:** The GRUB password rule (`grub2_uefi_password`) consistently failed because the VM lacked the `grub2-mkpasswd-pbkdf2` binary. Furthermore, when models correctly generated interactive password prompts, the non-interactive test harness could not supply input, resulting in timeouts instead of clean failures.
- **Upstream Reference Bugs:** Several `systemd-journal-upload` rules failed universally because the underlying OpenSCAP/SSG reference content contains a bug. It uses a regex fragment (a parenthesised alternation) directly within a `grep`/`sed` target path, which no shell can resolve.

- **Evaluation Adjustment:** These infrastructure and upstream failures were carefully isolated and excluded so they would not artificially lower a model's remediation score.

9 Discussion and Next Steps

- **Core Conclusion:** The validation stages demonstrate that a compact, locally hosted model (Qwen 2.5 7B) can reliably automate the tedious, rule-by-rule judgement calls and script generation that previously required a human security expert.
- **The Value of Guardrails:** The system succeeds by balancing automation with safety, handling the bulk of the remediation seamlessly while strictly routing high-risk or ambiguous decisions back to the user.
- **Future Work:** Moving forward, the main priority is system safety and improving the model's ability to successfully fix complex rules. The next steps involve testing the pipeline across a wider variety of Linux distributions and system roles to find its limits. The focus will be on helping the model write better scripts for the rules it currently fails on, while ensuring that critical safeguards, like human approvals for risky changes are never bypassed.